Martin Jonas A. Aranzado **07/29/26**
Assignment #1

1. What are the differences between Native, Hybrid, and Cross-platform mobile application development, and when is each approach commonly used?

These three are common approaches to mobile application development. The main thing they differ in is their performance, development process, code reuse, and access to device features.

a) **Native**
- these are apps that are built specifically for one operating system using its official programming languages and development tools (i.e. Android or IOS)
- It is commonly used when the app requires maximum performance, high security, or full access to device features
- It is ideal for apps that need the best user experience on a specific platform

b) **Hybrid**
- These are web applications packaged inside a native controller, which in turn allows it to run on multiple platforms and across multiple operating systems
- Really commonly used when the goal is fast development, with a simple functionality, and comes at a low cost
- It is often used for prototypes, internal business apps, and even content-based apps

c) **Cross-Platform**
- Cross-platform development uses frameworks that compile or render a shared codebase into apps for Android and iOS.
- When developers want to build for both Android and iOS using one codebase while maintaining good performance.
- Suitable for most business and consumer applications.

2. What is the Flutter Ecosystem, and how does it help developers create modern mobile applications?

Flutter Ecosystem
- is a collection of tools, frameworks, libraries, etc. that is provided by Google and the Flutter community that help developers build, test, and deploy applications efficiently.
- is a complete development environment that combines the Flutter SDK, Dart language, widgets, packages, development tools, and services like Firebase.
- It enables developers to create **high-quality, modern applications** for Android, iOS, web, desktop, and even embedded devices using a **single codebase**.

3. What is the Software Development Life Cycle (SDLC), and why is it important in building successful applications?

Software Development Life Cycle

- the SDLC is a structured process used to plan, design, develop, test, deploy, and maintain software applications.
- it provides a systematic approach that helps development teams create high-quality software efficiently while meeting user and business requirements.
- This structured process comes in different phases and follows these accordingly: **PLANNING, REQUIREMENT ANALYSIS, SYSTEM DESIGN, IMPLEMENTATION, TESTING, DEPLOYMENT,** and **MAINTENANCE.**
- this is considered to be important in developing successful applications because it provides a clear development process by organizing a project into manageable phases which also helps reduce development risks as it allows problems and issues to be detected early on in the making

4. How do Flutter development practices and the SDLC work together in creating real-world mobile applications?

Flutter development practices and the **Software Development Life Cycle (SDLC)** complement each other by combining a structured development process with powerful tools for building cross-platform applications.

- Flutter supports the SDLC by enabling efficient development, testing, and deployment through features like a single codebase, Hot Reload, and built-in tools.
- The SDLC provides a structured process that ensures Flutter applications are well-planned, properly tested, and meet user and business requirements.
- Together, Flutter and the SDLC help developers build high-quality, cost-effective, and maintainable mobile applications more quickly and reliably.

5. Why should developers understand these concepts before starting an application project?

- They help developers choose the right approach and tools (e.g., native, hybrid, or Flutter) based on the project's needs.
- They provide a clear development process through the SDLC, reducing errors, delays, and unnecessary costs.
- They improve the quality and success of the application by ensuring it is efficient, reliable, maintainable, and meets user requirements.